

Introdução a Algoritmos

Aula 14 — Recursividade

Raoni F. S. Teixeira · 2s/2026

Objetivos desta aula

Ao final desta aula, você deverá ser capaz de:

1. Identificar o caso base e o caso recursivo de uma função recursiva, e verificar que o argumento decresce a cada chamada;
2. Rastrear a pilha de chamadas de uma função recursiva simples, descrevendo os frames empilhados na descida e os resultados propagados na subida;
3. Implementar e comparar a exponenciação linear $O(n)$ e a exponenciação rápida $O(\lg n)$, explicando por que guardar o resultado intermediário é essencial na versão eficiente;
4. Explicar o risco de stack overflow em recursão sem caso base e relacioná-lo ao modelo de stack da Aula 13a.

1 Uma função que chama a si mesma

Nas aulas de funções, aprendemos que uma função pode chamar outras funções. Há um caso especial que não exploramos: uma função que chama **a si mesma**. Isso é recursão.

À primeira vista parece circular — e de fato é, se não houver um critério de parada. Mas com o critério certo, a recursão é uma ferramenta poderosa para expressar soluções que têm estrutura naturalmente recursiva.

A ideia central é sempre a mesma: um problema de tamanho n é resolvido em termos de um problema menor, de tamanho $n-1$ (ou $n/2$, ou outro subproblema), até chegar num caso tão pequeno que a resposta é trivial.

Definição — Função recursiva

Uma função recursiva é aquela que se chama direta ou indiretamente. Toda função recursiva correta tem dois componentes obrigatórios:

- **Caso base:** uma ou mais entradas para as quais a resposta é conhecida diretamente, sem nova chamada recursiva.
- **Caso recursivo:** a chamada à própria função com um argumento **estritamente menor**, que garante a convergência para o caso base.

2 Fatorial: o exemplo canônico

O fatorial de n é definido matematicamente de forma recursiva:

$$n! = \begin{cases} 1 & \text{se } n = 0 \\ n \times (n-1)! & \text{se } n > 0 \end{cases}$$

A tradução para C é direta:

```
int fatorial(int n) {
    if (n == 0)          /* caso base */
        return 1;
    return n * fatorial(n - 1); /* caso recursivo */
}
```

Isso compila, executa e produz a resposta correta. Mas o que acontece **na memória** quando chamamos `fatorial(4)`?

2.1 Recursão e a stack

Na Aula 13a aprendemos que cada chamada de função empilha um novo frame na stack com suas variáveis locais e parâmetros. Recursão não é diferente — cada chamada recursiva empilha um novo frame, e os frames se acumulam até o caso base ser atingido.

Para `fatorial(4)`:

Descida – empilhando	Subida – retornando
fatorial(4) aguarda	fatorial(4) = 4 x 6 = 24
fatorial(3) aguarda	fatorial(3) = 3 x 2 = 6
fatorial(2) aguarda	fatorial(2) = 2 x 1 = 2
fatorial(1) aguarda	fatorial(1) = 1 x 1 = 1
fatorial(0) – BASE	fatorial(0) = 1

Leia a coluna esquerda de baixo para cima (empilhamento) e a direita de baixo para cima (propagacao dos resultados). O caso base esta na base da pilha; o resultado final emerge no topo.

A execução tem duas fases: a **descida** (chamadas recursivas empilhando frames até o caso base) e a **subida** (retornos propagando resultados de volta até o chamador original).

Atenção

Recursão sem caso base — ou com caso base nunca alcançado — empilha frames indefinidamente até esgotar a stack. O programa termina com **stack overflow**. Esse é o análogo recursivo do loop infinito.

```
int f(int n) {
    return n * f(n - 1); /* sem caso base – stack overflow */
}
```

3 Exponenciação ingênua: $O(n)$

Calcular b^n de forma recursiva segue a mesma estrutura do fatorial:

$$b^n = \begin{cases} 1 & \text{se } n = 0 \\ b \times b^{n-1} & \text{se } n > 0 \end{cases}$$

```
double potencia_linear(double b, int n) {
    if (n == 0)
        return 1.0;
    return b * potencia_linear(b, n - 1);
}
```

Cada chamada reduz n em 1 e faz uma multiplicação. Para calcular b^n , são feitas exatamente n chamadas recursivas e n multiplicações.

Dizemos que este algoritmo tem **complexidade $O(n)$** : o número de operações cresce linearmente com n . Para $b^{\{1000\}}$, são necessárias 1000 multiplicações.

Complexidade

A notação $O(f(n))$ – le-se “O grande de f de n ” – descreve como o custo de um algoritmo cresce com o tamanho da entrada. $O(n)$ significa que o custo cresce proporcionalmente a n ; $O(\lg n)$ significa que cresce proporcionalmente ao logaritmo de n . Não se preocupe com a definição formal agora – o importante é a intuição: $O(\lg n)$ cresce **muito** mais devagar que $O(n)$, que cresce muito mais devagar que $O(n^2)$.

4 Exponenciação rápida: $O(\lg n)$

Existe uma observação simples que transforma o algoritmo:

$$b^n = \begin{cases} 1 & \text{se } n = 0 \\ (b^{n/2})^2 & \text{se } n \text{ é par} \\ b \times (b^{n/2})^2 & \text{se } n \text{ é ímpar} \end{cases}$$

Em vez de reduzir n em 1 a cada passo, reduzimos n **pela metade**. Isso é a diferença entre $O(n)$ e $O(\lg n)$.

```
double potencia_rapida(double b, int n) {
    if (n == 0)
        return 1.0;

    double meio = potencia_rapida(b, n / 2); /* calcula b^(n/2) */

    if (n % 2 == 0)
        return meio * meio; /* b^n = (b^(n/2))^2 */
    else
        return b * meio * meio; /* b^n = b * (b^(n/2))^2 */
}
```

A variável `meio` é crucial: ela guarda o resultado de `potencia_rapida(b, n/2)` para que não seja calculado duas vezes. Chamar `potencia_rapida(b, n/2)` diretamente em `meio * meio` sem guardá-la chamaria a função **duas vezes**, dobrando o trabalho e perdendo a vantagem do algoritmo.

4.1 Comparando os dois algoritmos

Para calcular $2^{\{10\}}$:

potencia_linear(2,10)	potencia_rapida(2,10)
10 chamadas, 10 multiplicações	4 chamadas, 4 multiplicações
pot(2,10) n=10	pot(2,10) n=10
pot(2,9) n=9	pot(2,5) n=5
pot(2,8) n=8	pot(2,2) n=2
...	pot(2,1) n=1
pot(2,1) n=1	pot(2,0) n=0 – BASE
pot(2,0) n=0 – BASE	

Para $n = 1,000,000$: a versão linear faz um milhão de multiplicações; a versão rápida faz apenas 20 (pois $\log_2 10^6 \approx 20$). Para $n = 10^{18}$ — um número que aparece em criptografia — a diferença é entre 10^{18} e 60 operações.

5 Recursão vs. iteração

Toda solução recursiva pode ser reescrita de forma iterativa, e vice-versa. A escolha depende de qual forma expressa melhor a estrutura do problema.

```
/* Fatorial iterativo */
int fatorial_iter(int n) {
    int resultado = 1;
    for (int i = 1; i <= n; i++)
        resultado *= i;
    return resultado;
}

/* Exponenciacao rapida iterativa */
double potencia_rapida_iter(double b, int n) {
    double resultado = 1.0;
    while (n > 0) {
        if (n % 2 == 1)
            resultado *= b;
        b = b * b;
        n /= 2;
    }
    return resultado;
}
```

A versão iterativa da exponenciação rápida é menos intuitiva que a recursiva — o algoritmo tem estrutura naturalmente recursiva (dividir pela metade), e a versão recursiva espelha essa estrutura diretamente.

Em geral, prefira recursão quando o problema tem estrutura recursiva clara (árvores, divisão e conquista, backtracking) e prefira iteração quando a recursão seria profunda o suficiente para pressionar a stack ou quando a versão iterativa é igualmente legível.

Exemplo — Busca binária recursiva

Na Aula 9 propusemos busca binária como desafio. A versão recursiva torna a estrutura do algoritmo explícita:

```
/* Busca binaria em v[esq..dir], retorna indice ou -1 */
int busca_bin(int v[], int esq, int dir, int alvo) {
    if (esq > dir)
        return -1; /* caso base: intervalo vazio */

    int meio = (esq + dir) / 2;

    if (v[meio] == alvo)
        return meio; /* caso base: encontrado */
    else if (alvo < v[meio])
        return busca_bin(v, esq, meio-1, alvo); /* metade esquerda */
    else
        return busca_bin(v, meio+1, dir, alvo); /* metade direita */
}
```

Cada chamada elimina metade dos candidatos. Para $n = 1000$ elementos, no pior caso são $\lceil \log_2 1000 \rceil = 10$ chamadas.

6 O horizonte: para onde este curso aponta

Esta é a última aula. Ao longo do curso, você construiu os fundamentos que tornam possível aprender qualquer coisa em computação — não apenas em C, mas em qualquer sistema onde a máquina precise ser entendida.

O que você domina agora:

As cinco peças fundamentais (variável, sequência, decisão, repetição, função), o modelo de memória completo (código, dados, stack, heap), tipos compostos (vetores, matrizes, strings, structs), passagem por valor e por referência, alocação dinâmica e recursão. Não são conceitos isolados — são um vocabulário coerente para pensar sobre o que um computador faz.

Cada disciplina que vem a seguir usa esse vocabulário como ponto de partida:

Estruturas de dados e algoritmos

Você já conhece vetores e buscas simples. O próximo passo são estruturas que organizam dados de forma a tornar certas operações mais eficientes: listas encadeadas (cada elemento aponta para o próximo — ponteiros aplicados recursivamente), árvores binárias (cada nó tem dois filhos — recursão aplicada à estrutura), tabelas hash (acesso em $O(1)$ amortizado). A base é malloc e struct com ponteiro para si mesma:

```
typedef struct No {
    int valor;
    struct No *esq;
    struct No *dir;
} No;
```

Esse é o nó de uma árvore binária. Você já sabe declarar isso.

Complexidade de algoritmos

Nesta aula vimos $O(n)$ vs $O(\lg n)$ de forma intuitiva. A disciplina de análise de algoritmos formaliza essa intuição: notação assintótica, casos melhor/médio/pior, classes de complexidade (P, NP). A pergunta central é “quão devagar este algoritmo cresce?” — e você já tem os exemplos concretos para ancorar essa teoria: busca linear é $O(n)$, busca binária é $O(\lg n)$, ordenação por seleção é $O(n^2)$, produto de matrizes ingênuo é $O(n^3)$.

Sistemas operacionais e threads

Na Aula 13a vimos stack e heap como regiões de memória de um processo. Um sistema operacional gerencia múltiplos processos simultaneamente — cada um com seu próprio espaço de memória. **Threads** são linhas de execução dentro do mesmo processo, compartilhando o heap mas com stacks separadas. O problema central é a **condição de corrida**: duas threads modificando a mesma variável do heap simultaneamente produzem resultados imprevisíveis. Mutexes e semáforos são as ferramentas para evitar isso — mecanismos de sincronização que você entenderá muito mais rapidamente porque já sabe o que é stack, heap e ponteiro.

Microcontroladores e sistemas embarcados

C é a linguagem dominante em microcontroladores (Arduino, STM32, ESP32). O código que você escreveu neste curso roda, com pequenas adaptações, em hardware com 2 KB de RAM e sem sistema operacional. A diferença: não há malloc (sem heap gerenciado), não há printf para tela, e cada byte importa. O modelo de memória que aprendemos — especialmente a distinção entre stack e dados estáticos — é ainda mais crítico quando a stack total tem 512 bytes. Registradores, interrupções e DMA são os próximos conceitos — mas todos se apoiam em ponteiros, structs e a ideia de que memória é um array de bytes numerados.

Linguagens de alto nível

Python, Java, JavaScript — todas gerenciam memória automaticamente (garbage collector) e escondem ponteiros. Mas elas **implementam** vetores, dicionários e objetos usando exatamente as estruturas que você conhece. Saber C torna você um programador mais consciente em qualquer linguagem: você entende por que copiar uma lista em Python com `b = a` não cria uma cópia independente, por que strings são imutáveis, por que passar um objeto para uma função não passa uma cópia. São as mesmas questões de valor vs. referência que estudamos nas Aulas 7 e 8.

7 Exercícios finais

1. Escreva uma função recursiva `int soma_digitos(int n)` que retorna a soma dos dígitos de um inteiro positivo. Por exemplo, `soma_digitos(1234)` retorna 10. (Dica: o último dígito é $n \% 10$; os demais formam $n / 10$.) Identifique o caso base e o caso recursivo antes de implementar.

$$F(0) = 0,$$

$$F(1) = 1,$$

2. Implemente a sequência de Fibonacci recursivamente: $F(n) = F(n-1) + F(n-2)$. Meça (contando chamadas) quantas vezes `F(2)` é calculado ao computar `F(10)`. O que isso sugere sobre a eficiência desta implementação?
3. Escreva uma função recursiva `void imprime_binario(int n)` que imprime a representação binária de `n` **sem usar vetor auxiliar**. (Dica: imprima primeiro `imprime_binario(n/2)`, depois $n \% 2$. Por que essa ordem produz os bits da esquerda para a direita?)

4. Escreva `int potencia_rapida_iter(int b, int n)` — a versão iterativa da exponenciação rápida apresentada na aula. Verifique que produz os mesmos resultados que a versão recursiva para os pares (2,10), (3,5), (5,8), (2,20).
5. (*Desafio*) Escreva uma função recursiva `void permutacoes(char s[], int inicio)` que imprime todas as permutações da string `s`. A ideia: fixe `s[inicio]` e permuta o resto recursivamente; depois troque `s[inicio]` com cada `s[i]` para `i > inicio` e repita. Quantas permutações tem uma string de comprimento `n`?

8 Encerramento

Sobre o que foi construído aqui

Programar não é memorizar sintaxe — é desenvolver um modo de pensar. Ao longo deste curso você aprendeu a decompor um problema em partes menores, a expressar cada parte com precisão suficiente para que uma máquina a execute, e a raciocinar sobre o que acontece quando a execução diverge do esperado.

Esses hábitos — decompor, precisar, rastrear — transcendem qualquer linguagem ou ferramenta. C foi o veículo porque ela não esconde nada: memória, ponteiros, tipos, tempo de vida de variáveis — tudo é explícito e verificável. Aprender em C é aprender com o capô aberto.

O que vem a seguir será mais abstrato, mais poderoso e, às vezes, mais confortável. Mas quando algo não funcionar — e não vai funcionar, em qualquer linguagem, em qualquer nível — você terá o modelo certo para procurar a causa: onde este dado vive na memória? quem é responsável por ele? o que acontece quando esta função retorna?

Bom trabalho. Agora vá construir algo.